

ASSIGNMENT 2: MODEL STEALING DETECTION

Anonymous authors

Paper under double-blind review

Varun Chaturvedi (7087955), Vaishnavi Anil Borse (7075846), Team 50

Github code: <https://github.com/arcnova2095/TML-adveserial-attacks/blob/main/task3.ipynb>

1 ASSIGNMENT

In this assignment we have trained a ResNet-50 classifier on a 9-class image dataset to be robust against the adversarial examples. Our approach is a standard adversarial training: for each batch we have generated PGD adversarial examples on the fly and train on clean and adversarial images together. After trying a few different combinations of attacks and architectures, the clean + PGD setup with ResNet-50 and 10 PGD steps gave the best results, reaching a composite validation score of 0.5885 (79.5% clean accuracy, 38.2% PGD accuracy) before early stopping at epoch 21.

2 METHODOLOGY

We implemented two attacks.

FGSM. Fast Gradient Sign Method is a single-step attack. It computes the gradient of the loss with respect to the input pixels and takes it one step in the direction that hurts most:

$$x_{\text{adv}} = x + \varepsilon \cdot \text{sign}(\nabla_x \mathcal{L}(f(x), y))$$

It is fast but weak; a single step often does not fully exploit the allowed perturbation budget.

PGD. Projected Gradient Descent is the iterated version. It starts from a random point inside the ε -ball around the input and takes multiple gradient steps that projects back after the each one:

$$x^{(t+1)} = \Pi_{B_\varepsilon(x)} \left(x^{(t)} + \alpha \cdot \text{sign}(\nabla_x \mathcal{L}(f(x^{(t)}), y)) \right)$$

with step size $\alpha = 2/255$ and 10 steps in our final setup. Because it iterates and it finds significantly stronger adversarial examples than FGSM with the same ε budget.

2.1 ADVERSARIAL TRAINING APPROACH

The main idea of the adversarial training is pretty straightforward: generate the adversarial examples during the training and include them in each batch so that the model learns to be correct under attack too. Concretely, for every training batch we:

1. Run PGD on the batch using the *current* model to produce the adversarial examples.
2. Concatenate the original clean images and the adversarial images (doubling the batch size).
3. Do a forward pass on this combined batch, compute cross-entropy, and update the weights.

The fact that adversarial examples are generated fresh every epoch using the current model is important, the model is always being challenged by its own current blind spots rather than a fixed set of precomputed perturbations.

WHY NOT FGSM?

We experimented by substituting or mixing in the FGSM-generated examples, and it continuously made things worse. The intuition is that the FGSM examples are just weaker, and the training on weak adversarial examples does not prepare the model for a stronger PGD evaluation. Including them ends up diluting the training signal.

WHY RESNET-50?

We tried the ResNet-18, ResNet-34, and ResNet-50. ResNet-18 tended to overfit. The training accuracy has climbed nicely but the validation robustness did not improved. ResNet-34 was better but still not the best. ResNet-50 turned out to be the right capacity for this problem. One thing we noticed is that the adversarial training seems to demand more model capacity than the standard training. This is probably because the model needs to simultaneously represent clean features and be invariant to the whole family of perturbations around every input. The larger model handled this better.

TRAINING STRATEGY

We ran the below four combinations :

Strategy	Val score	Observation
Clean + random(FGSM or PGD)	lower	Mixing weak attacks hurts the consistency
Random(clean, FGSM, or PGD)	lower	Too noisy, the model doesn't commit to either
Clean + FGSM + PGD	slightly lower	FGSM still drags the things down
Clean + PGD	best	Focused, consistent and evaluates well

Table 1: Comparison of the training strategies. All variants used ResNet-50 and PGD-10.

Every time the FGSM was included, the combined score dropped. The clean + PGD combination was the most consistent and worked best at the evaluation time, which makes sense since the evaluation attack is PGD.

EFFECT OF PGD STEPS DURING TRAINING

With the ResNet-50, using 10 PGD steps during training outperformed the 7 steps. This is worth noting because the relationship is not always simple, with the smaller models like ResNet-34, more PGD steps during training actually made things worse, probably because the inner attack becomes too strong for the model to learn from. With more capacity (ResNet-50), the model can keep up with stronger perturbations and benefits from the hard training signal.

2.2 SUMMARY OF RESULTS

The best checkpoint was saved at epoch 16 with the following validation results:

Metric	Value
Clean accuracy	79.5%
PGD accuracy (10 steps, $\epsilon = 8/255$)	38.2%
Composite score	0.5885
Best epoch	16 of 21

Table 2: Final model performance on the 1,000-sample validation set.

Training stopped in the epoch 21 after five consecutive epochs without improvement. We noticed in the training logs that from epoch 16 to 21 the clean accuracy actually kept going up (it hit 80.5% at epoch 21), but the PGD accuracy dropped from 38.2% to 35.9%. This is an example of the accuracy robustness tradeoff, the model was becoming less robust as it continued to overfit the clean distribution. Early stopping caught this correctly and the epoch 16 checkpoint has the better balance.

A 38% robust accuracy under a 10-step PGD attack with $\epsilon = 8/255$ is better than a model which would collapse to near 0% under the same attack. The clean accuracy of 79.5% on a 9-class problem shows that the model did not sacrifice too much normal performance in exchange for robustness.

3 CONCLUSION

The setup that worked best was adversarial training with ResNet-50 using clean images and adversarial examples generated by PGD in each batch (10 steps, $\varepsilon = 8/255$, $\alpha = 2/255$). FGSM continued to hurt when included and was dropped entirely. ResNet-50 had the right capacity, smaller models overfitted or lacked robustness, larger models weren't tried.

There is definitely room to improve further. Longer trainings with the decaying learning rate, data augmentation would likely push the composite score higher. But the current results already shows that straightforward PGD-based adversarial training gives a meaningful robustness gain.